

南开大学
计算机大类

高级语言程序设计
实验报告

作业题目： Slacker——基于 UE5 的 3D 潜行类游戏
姓 名： 李佩霖
学 号： 2510547

2026 年 5 月 14 日

目录

1	作业题目	2
2	开发软件	2
3	课题要求	2
4	主要流程	3
4.1	整体流程	3
4.2	AI 状态机算法	4
4.2.1	巡逻状态 (Patrol)	4
4.2.2	追逐状态 (Chase)	5
4.2.3	押送状态 (Escort) ——核心难点	5
4.3	动画系统	7
4.4	UI 系统	8
5	单元测试	9
5.1	测试案例设计	9
5.2	关键 Bug 修复记录	9
6	收获	10
6.1	AI 状态机设计	10
6.2	物理碰撞与移动	10
6.3	动画系统与根运动	10
6.4	C++ 与引擎架构	11
6.5	版本管理与项目备份	11
6.6	调试方法论	11

1. 作业题目

Slacker——基于 Unreal Engine 5 的 3D 办公室潜行类游戏。

在游戏中，玩家扮演一名办公室职员，目标是在不被老板发现的情况下完成 5 次”摸鱼”行为以通关。老板 AI 会在办公室内巡逻，一旦检测到玩家就会开始追逐，追上后将玩家押送回座位进行 3 秒的”思想教育”（惩罚时间）。玩家需要利用地形和时机，在 5 个不同的”摸鱼点”（SlackPoint）依次完成摸鱼操作，全部完成后即可获胜。

本项目的核心特色包括：基于 UE5 行为树的多状态 AI 系统（巡逻/追逐/押送三态切换）、完整的动画蒙太奇系统（走/跑/坐/被扛四种状态）、物理碰撞优化解决方案、以及 UMG 进度条 UI 系统。

2. 开发软件

表 1 开发环境一览

软件名称	版本	用途
Unreal Engine	5.4	游戏引擎、场景搭建、蓝图系统
Visual Studio	2022	C++ 代码编写与调试
Blender	4.1	角色模型尺寸调整
Adobe Photoshop	2024	UI 素材制作
Git	2.43	版本管理与备份
GitHub Desktop	3.3	Git 图形化操作

3. 课题要求

本项目为高级语言程序设计 2-2 课程大作业，要求运用面向对象编程思想，结合游戏引擎开发一个具有完整玩法的 3D 游戏。具体要求如下：

- 面向对象设计：**使用 C++ 类继承体系设计玩家角色、AI 角色、交互物体等游戏实体；
- AI 行为系统：**实现具有多种状态的人工智能，包含巡逻、追逐、押送等状态切换逻辑；
- 动画系统：**为角色配置多种动画状态（走、跑、坐、被扛起），并实现平滑的状态切换；
- 碰撞与物理：**正确处理角色间的物理碰撞关系，避免 AI 或玩家卡死；
- UI 交互：**设计 HUD 界面，显示游戏倒计时和惩罚进度条；
- 关卡设计：**搭建完整的办公室场景，合理放置巡逻点、摸鱼点和座位。

4. 主要流程

4.1 整体流程

游戏的整体核心循环围绕 BOSS AI 的三状态行为机展开，同时配合玩家的摸鱼进度判定胜负。整体架构如图 1 所示。

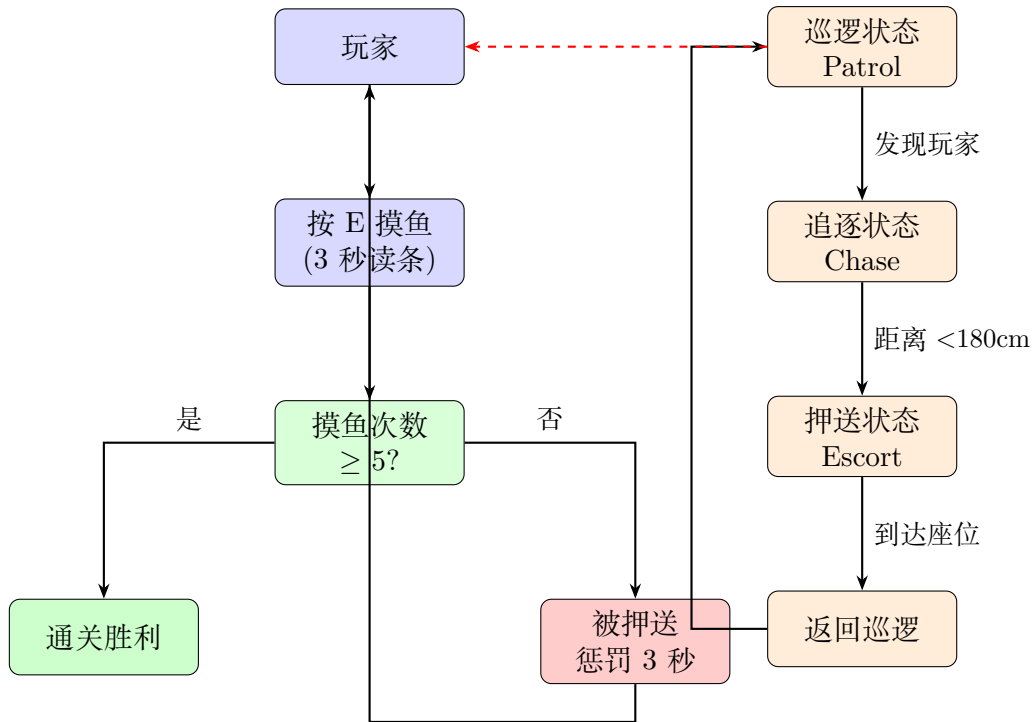


图 1 游戏整体流程图

实现思路：定义 BOSS AI 的三个核心状态——**Patrol**（巡逻）、**Chase**（追逐）、**Escort**（押送）。通过 UE5 的 **AAIController** 配合行为树实现状态切换。玩家角色通过 **ACharacter** 基类派生，利用 **CharacterMovement** 组件控制移动。摸鱼点（SlackPoint）和座位（Seat）均继承自 **AActor**，通过碰撞体触发交互。

核心类的继承关系如图 2 所示。

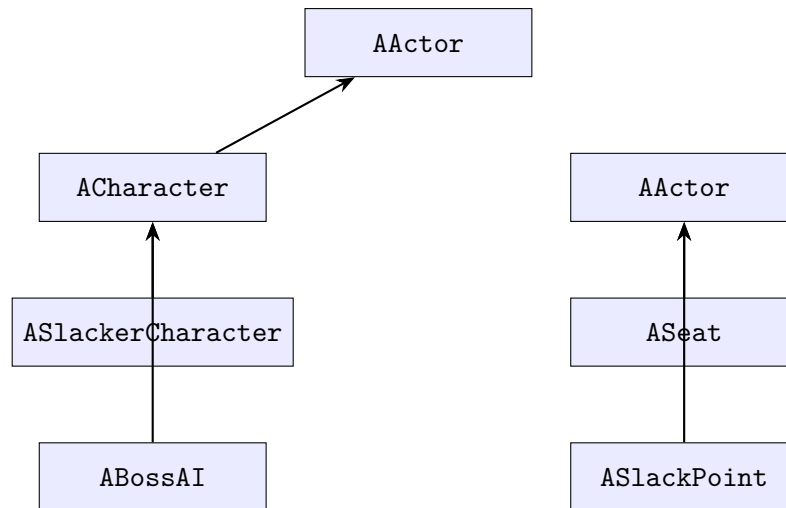


图 2 核心类继承关系图

4.2 AI 状态机算法

BOSS AI 的核心是三状态自动机，状态转换逻辑如下：

4.2.1 巡逻状态 (Patrol)

BOSS 在预设的巡逻点 (PatrolPoint) 之间循环移动。使用 `AAIController::MoveToActor()` 进行路径规划，到达一个点后等待 2 秒，再前往下一个点。

```
1 void ABossAI::PatrolLogic()
2 {
3     if (!bPatrolWaiting)
4     {
5         // 移动到下一个巡逻点
6         AAIController->MoveToActor(
7             PatrolPoints[CurrentPatrolIndex],
8             50.0f // 接受半径
9         );
10        CurrentPatrolIndex =
11            (CurrentPatrolIndex + 1) % PatrolPoints.Num();
12        bPatrolWaiting = true;
13    }
14 }
```

Listing 1: 巡逻状态核心逻辑

4.2.2 追逐状态 (Chase)

当 BOSS 的 PawnSensing 组件检测到玩家时，切换到 Chase 状态。此时 BOSS 以跑步速度 (800 cm/s) 追逐玩家。同时触发玩家的逃跑动画——将玩家速度提升至 1000 cm/s 并播放 Run 蒙太奇。

```

1 void ABossAI::OnSeePlayer(APawn* Pawn)
2 {
3     if (CurrentState == EBossState::Chase) return;
4
5     CurrentState = EBossState::Chase;
6     GetCharacterMovement()->MaxWalkSpeed = 800.0f;
7
8     // 触发玩家逃跑
9     if (ASlackerCharacter* Player =
10         Cast<ASlackerCharacter>(Pawn))
11     {
12         Player->GetCharacterMovement()
13             ->MaxWalkSpeed = 1000.0f;
14         Player->PlayAnimMontage(PlayerRunMontage);
15     }
16 }

```

Listing 2: 追逐状态触发

4.2.3 押送状态 (Escort) ——核心难点

当 BOSS 与玩家距离小于 180cm 时，进入 Escort 状态。这是整个项目中最复杂也是最具挑战性的部分。

问题分析：最初使用 MoveToActor() 让 BOSS 押送玩家到座位，但 BOSS 在抓到玩家后频繁卡死。经过深入排查，发现问题根源是：

1. 玩家胶囊体与 BOSS 胶囊体重叠导致物理碰撞冲突；
2. 每帧调用 MoveToActor() 导致路径不断重置；
3. StopMovement() 与后续的移动指令产生冲突。

解决方案：

1. 禁用玩家碰撞：Player->SetActorEnableCollision(false);
2. 将玩家放置在 BOSS 身前：ForwardVector × 60，Z 轴抬高 80；
3. 改用 AddMovementInput() 直接控制移动，避免 NavMesh 路径规划的不可控行为；

4. 添加 10 秒超时保护：若 10 秒内未到达座位，直接 Teleport 到目标位置。

```
1 void ABossAI::EscortLogic()
2 {
3     if (!TargetSeat) return;
4
5     // 计算到座位的方向
6     FVector SeatLocation = TargetSeat->GetActorLocation();
7     FVector Direction = SeatLocation - GetActorLocation();
8     Direction.Z = 0;
9     float Distance = Direction.Size();
10
11     if (Distance < 150.0f)
12     {
13         // 到达座位，执行惩罚
14         TargetSeat->ForceSit(CarriedPlayer);
15         ReturnToPatrol();
16         return;
17     }
18
19     // 使用 AddMovementInput 直接移动
20     Direction.Normalize();
21     AddMovementInput(Direction, 1.0f);
22
23     // 同步玩家位置到 BOSS 身前
24     FVector CarryPos = GetActorLocation()
25         + GetActorForwardVector() * 60.0f
26         + FVector(0, 0, 80);
27     CarriedPlayer->SetActorLocation(CarryPos);
28
29     // 10秒超时检测
30     if (GetWorld()->GetTimeSeconds()
31         - EscortStartTime > 10.0f)
32     {
33         // 强制传送
34         CarriedPlayer->SetActorLocation(SeatLocation);
35         TargetSeat->ForceSit(CarriedPlayer);
36         ReturnToPatrol();
37     }
38 }
```

Listing 3: 押送状态核心逻辑

4.3 动画系统

角色动画采用**动画蒙太奇 (AnimMontage)** 方案，通过 C++ 在 Tick 中检测状态并播放对应动画。所有 Mixamo 导入的动画均关闭了 `EnableRootMotion`，避免根运动导致的位置偏移。

表 2 动画状态对应表

角色	状态	动画蒙太奇	触发条件
BOSS	巡逻走	WalkMontage	<code>CurrentState == Patrol</code>
BOSS	追逐跑	RunMontage	<code>CurrentState == Chase</code>
BOSS	押送	CarryMontage	<code>CurrentState == Escort</code>
玩家	走路	WalkMontage	速度 > 0 且非跑步
玩家	跑步	RunMontage	BOSS 开始追逐
玩家	坐下	SitMontage	被强制坐下
玩家	被扛	CarriedMontage	被 BOSS 押送中

BOSS 动画更新逻辑：

```

1 void ABossAI::UpdateBossAnimation()
2 {
3     UAnimInstance* AnimInst = GetMesh()
4         ->GetAnimInstance();
5     if (!AnimInst) return;
6
7     switch (CurrentState)
8     {
9         case EBossState::Patrol:
10             if (!AnimInst->Montage_IsPlaying(
11                 WalkMontage))
12                 PlayAnimMontage(WalkMontage);
13             break;
14         case EBossState::Chase:
15             if (!AnimInst->Montage_IsPlaying(
16                 RunMontage))
17                 PlayAnimMontage(RunMontage);
18             break;
19         case EBossState::Escort:
20             if (!AnimInst->Montage_IsPlaying(
21                 CarryMontage))
22                 PlayAnimMontage(CarryMontage);
23             break;
24     }

```



```
25 }
```

Listing 4: BOSS 动画状态更新

4.4 UI 系统

使用 UMG (Unreal Motion Graphics) 创建 HUD, 包含两个核心元素:

1. **游戏计时器**: 从游戏开始计时, 显示在屏幕右上角, 格式为 MM:SS, 通关时停止;
2. **惩罚进度条**: 当玩家被押送到座位时, 显示 3 秒惩罚倒计时, 以绿色进度条形式呈现。

GameMode 的 Tick 函数负责更新 UI 显示:

```
1 void ASlackerGameMode::Tick(float DeltaTime)
2 {
3     Super::Tick(DeltaTime);
4
5     // 更新游戏总时间
6     if (!bGameWon)
7     {
8         TotalGameTime += DeltaTime;
9         UpdateTimerDisplay();
10    }
11
12    // 更新惩罚倒计时
13    if (bPenaltyActive)
14    {
15        PenaltyTimeRemaining -= DeltaTime;
16        if (PenaltyTimeRemaining <= 0)
17        {
18            StopPenaltyTimer();
19        }
20        UpdatePenaltyDisplay();
21    }
22 }
```

Listing 5: HUD 更新逻辑

5. 单元测试

5.1 测试案例设计

针对游戏的核心功能模块，设计了以下测试案例：

表 3 测试案例表

测试项	预期结果	实际结果
BOSS 巡逻路径	BOSS 按顺序访问所有巡逻点	通过
BOSS 视线检测	玩家进入检测范围触发追逐	通过
BOSS 追逐速度	速度为 800 cm/s	通过
玩家逃跑速度	速度提升至 1000 cm/s	通过
距离检测切换	<180cm 进入押送状态	通过
押送碰撞禁用	玩家 SetActorEnableCollision(false)	通过
押送位置同步	玩家在 BOSS 身前 (60, 0, 80)	通过
座位到达检测	<150cm 触发坐下	通过
惩罚时间	恰好 3 秒倒计时	通过
惩罚进度条显示	UI 进度条从满到空	通过
站起速度恢复	MaxWalkSpeed 恢复为 200	通过
摸鱼进度累加	每次 +1，达到 5 次触发胜利	通过
游戏计时器	从 0 开始累加，胜利时停止	通过
动画状态切换	状态改变时播放对应 Montage	通过
超时保护机制	10 秒未到达 Teleport 到座位	通过
NavMesh 寻路	办公室场景可正常寻路	通过

5.2 关键 Bug 修复记录

在开发过程中遇到了多个关键 Bug，修复过程如下：

表 4 关键 Bug 修复记录

Bug 描述	原因分析	修复方案
BOSS 抓人后卡死	MoveToActor 每帧调用导致路径重置	改用 AddMovementInput
BOSS 完全不动	bEscortMoving 标志设为 true 后永不重试	移除标志，改用时间戳
碰撞阻挡	玩家胶囊体与 BOSS 胶囊体重叠	禁用玩家碰撞
编译错误 C4819	中文注释含非 ASCII 字符	清理所有特殊字符
站起后仍跑步	StandUp 未恢复 MaxWalkSpeed	添加 MaxWalkSpeed=200
动画上升漂移	Mixamo 动画 EnableRootMotion	关闭所有 RootMotion
StopMovement 冲突	Stop 与 MoveToActor 指令冲突	移除 StopMovement 调用

6. 收获

6.1 AI 状态机设计

通过本次项目，深入理解了 AI 状态机的设计模式。BOSS 的三状态（Patrol、Chase、Escort）切换看似简单，但状态间的边界条件和转换逻辑十分微妙。

例如在 Chase 到 Escort 的转换中，距离阈值的选择（180cm）、接受半径的设置（150cm）都需要反复调试。最关键的是理解了 MoveToActor 的局限性——它是一个“到目的地”的指令，而不是“持续跟随”的指令，如果高频调用会导致不可预期的行为。

6.2 物理碰撞与移动

UE5 的 CharacterMovement 组件基于物理引擎，碰撞体的交互非常复杂。本项目中最深刻的教训是：两个 Character 的胶囊体重叠时会产生不可预期的物理冲突，导致角色被“弹飞”或“卡死”。通过 SetActorEnableCollision(false) 在押送期间禁用玩家碰撞，是一个看似简单但非常有效的解决方案。同时学会了使用 AddMovementInput() 进行直接移动控制，这种方式虽然失去了 NavMesh 的避障能力，但在特定场景下更加可靠可控。

6.3 动画系统与根运动

Mixamo 动画导入 UE5 后默认开启 EnableRootMotion，这会导致角色的实际位置由动画的根骨骼运动驱动，而非由代码控制。在项目中表现为播放走路动画时角色会不受控地移动甚至上升。通过在动画序列中关闭 EnableRootMotion，并使用

PlayAnimMontage() 在代码中控制动画播放，实现了动画表现与逻辑控制的完全分离。

6.4 C++ 与引擎架构

通过阅读 UE5 源码和 API 文档，对引擎的面向对象架构有了更深的理解：

- AActor 的生命周期管理 (Spawn/Destroy)；
- UActorComponent 的模块化设计思想；
- UAnimInstance 与蒙太奇的协同工作方式；
- AAIController 与行为树的分工；
- Tick 函数在游戏主循环中的执行时机。

6.5 版本管理与项目备份

项目中期曾因误操作导致代码文件损坏，幸亏使用了 Git 进行版本管理才得以恢复。后续养成了频繁提交的习惯，最终将项目完整备份到 GitHub。这深刻体会到了“提交越早，后悔越少”的道理。

6.6 调试方法论

面对“BOSS 卡死”这个核心 Bug，最初的思路是在状态切换逻辑中找问题，增加了各种标志位和冷却时间，反而让问题更复杂。后来通过在 Tick 中打印日志、观察 BOSS 和玩家的实时位置，才定位到物理碰撞是根本原因。这个经历说明：遇到 Bug 时，先观察现象（打印日志、可视化调试），再分析原因，最后动手修改，比盲目猜测要高效得多。

本次大作业通过开发“Slacker”游戏，完整实践了从游戏设计、C++ 编程、AI 算法实现、动画系统集成到 UI 界面开发的全部流程。项目最终成功打包为可执行文件 (.exe) 并上传至 GitHub 备份，达到了课程要求的全部目标。

AI 工具使用披露

本项目开发过程中使用了人工智能工具辅助完成部分工作，具体披露如下：

AI 工具	具体用途	使用位置/范围
Kimi (Moonshot AI)	代码开发与 Bug 修复	BOSS AI 的 Escort、Chase、Patrol 逻辑代码；碰撞禁用方案；动画系统 Montage 播放逻辑；站起速度恢复等核心代码
Kimi (Moonshot AI)	技术方案设计	AI 三状态机架构设计；押送状态 AddMovementInput 替代方案；10 秒超时保护机制
Kimi (Moonshot AI)	项目文档生成	游戏说明文档、视频介绍稿、AI 视频提示词（开头/结尾）、视频简介

表 5 AI 工具使用情况说明

声明：AI 工具在本项目中主要承担辅助编程与文档撰写的角色。所有游戏核心逻辑、算法设计思路、Bug 修复方案均经过本人的理解、验证与调整，最终代码在 Unreal Engine 5 中编译运行通过。本实验报告的内容基于项目实际开发过程的真实记录，AI 工具用于提升文档编写效率和代码结构规范性。